End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Name:	Pranav Menon
Roll No.:	24011101014
Degree & Branch:	B.Tech Artificial Intelligence & Data Science
Semester:	V
Subject Code & Name:	CS3807 – Deep Learning Laboratory
AY:	2026–27

Shiv Nadar University Chennai

1 Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Sequences are prepared, visualized, and used to train RNN/LSTM/GRU models; their performance is compared, convergence and generalization are analyzed, and the pipeline is extended to video understanding using CNN feature extraction followed by an LSTM or GRU. A small synthetic sequence-to-sequence task demonstrates the encoder–decoder framework.

2 Learning Outcomes

After completing this experiment, the student is able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3 Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI HAR dataset contains smartphone sensor measurements collected from subjects performing six activities: WALKING, WALKING_UPSTAIRS, WALKING_DOWNSTAIRS, SITTING, STANDING and LAYING. The raw inertial signal files are used (not only the pre-computed 561-feature vectors). Each activity window contains 128 temporal measurements; the three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels, giving the input representation

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

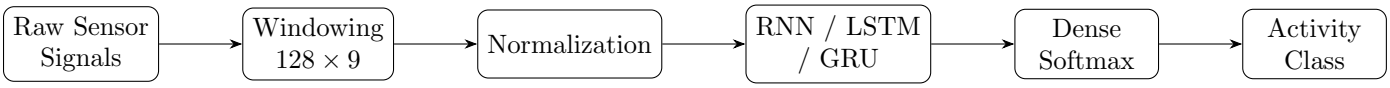
Dataset source: `https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones`

Recommended laboratory subset: the first 1500–3000 training windows, preserving all six classes with approximately balanced representation.

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Table 1: Input tensor dimensions.

4 Overall Experimental Pipeline



The three recurrent models use the same input, output layer, optimizer, batch size and evaluation protocol.

5 Preprocessing

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9.
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets (70%/15%/15%).
7. Verify the class distribution.

The test data is not used during model selection or hyperparameter tuning.

Preprocessing Output

```
Input tensor shape:
  Training   : (N_train, 128, 9)
```

```
  Validation : (N_val, 128, 9)
  Testing    : (N_test, 128, 9)
Number of classes: 6
Number of features per time step: 9
Sequence length: 128
```

6 Temporal Data Visualization

Plot 1: Sensor signal versus time – at least three representative channels, from at least three different activity classes.

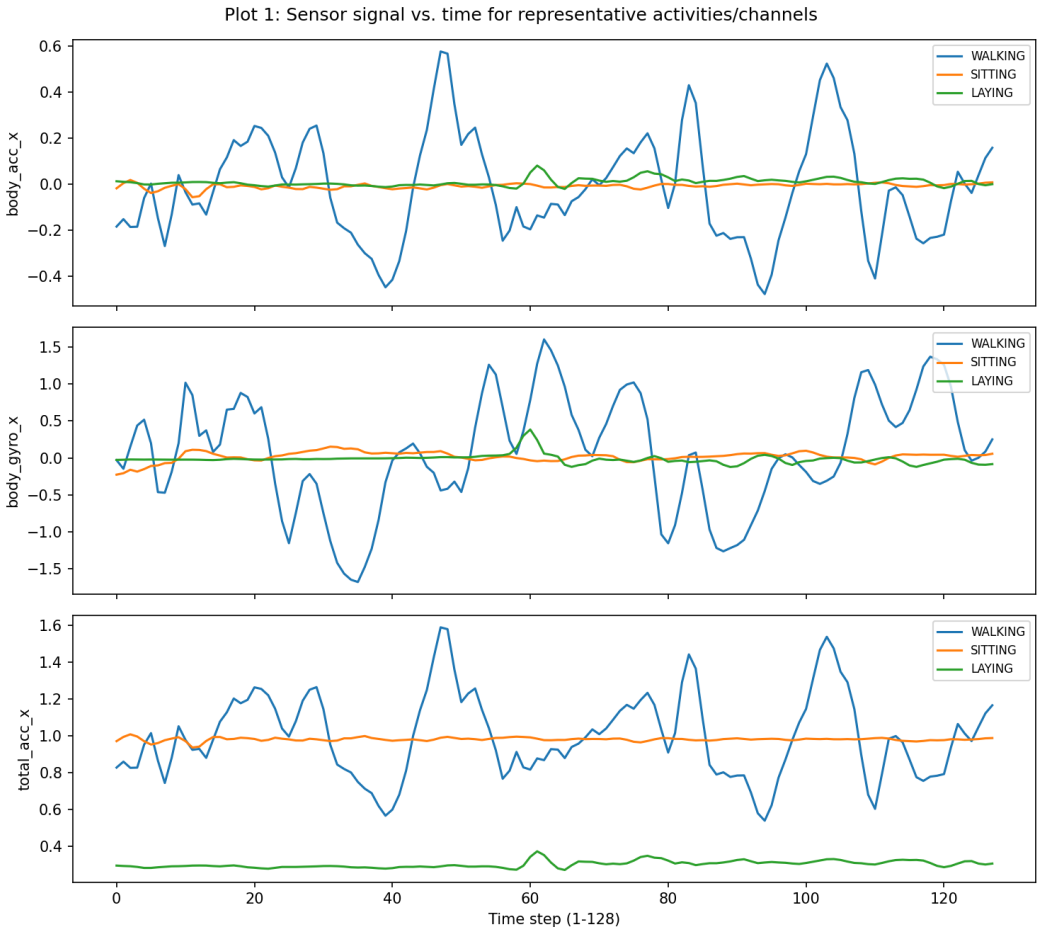


Figure 1: Sensor signal vs. time step for representative activities/channels.

7 Vanilla RNN Architecture

A Vanilla RNN maintains a hidden state updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h), \qquad y_t = g(W_y h_t + b_y).$$

For sequence classification, the final hidden representation is passed to a classifier. The same recurrent weights are reused across all time steps in the unrolled view.

8 Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states; BPTT propagates the error backward through the recurrent steps. Repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large:

$$\left\| \frac{\partial h_t}{\partial h_{t-k}} \right\| \to 0 \quad \text{(vanishing)}, \qquad \text{or very large (exploding)}.$$

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

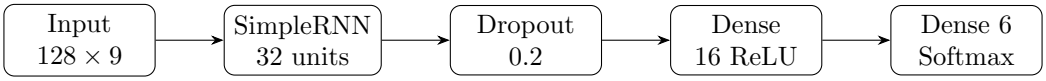
Numerical Exercise

Given $x_1 = 0.5,\ x_2 = 0.7,\ x_3 = 0.2,\ h_0 = 0,\ W_x = 0.5,\ W_h = 0.8,\ b = 0.1$ and $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$:

t	$z_t = W_x x_t + W_h h_{t-1} + b$	$h_t = \tanh(z_t)$	(program value)
1	*[fill in]*	*[fill in]*	*[fill in]*
2	*[fill in]*	*[fill in]*	*[fill in]*
3	*[fill in]*	*[fill in]*	*[fill in]*

Table 2: Manual BPTT calculation, compared with the program output.

9 RNN Implementation



Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

Table 3: Training configuration (identical for RNN, LSTM, GRU).

10 LSTM Architecture

LSTM introduces a memory cell and gates to control information flow:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad \text{(forget gate)} \tag{1}$$
$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad \text{(input gate)} \tag{2}$$
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad \text{(output gate)} \tag{3}$$
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C) \quad \text{(candidate cell state)} \tag{4}$$
$$C_t = f_t C_{t-1} + i_t \tilde{C}_t \quad \text{(cell state)} \tag{5}$$
$$h_t = o_t \tanh(C_t) \quad \text{(hidden state)} \tag{6}$$

11 GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z) \quad \text{(update gate)} \tag{7}$$
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r) \quad \text{(reset gate)} \tag{8}$$
$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h) \quad \text{(candidate hidden state)} \tag{9}$$
$$h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t \quad \text{(hidden state update)} \tag{10}$$

Comparison of gates/state representations: RNN has only a hidden state; LSTM adds a cell state, forget gate, input gate and output gate; GRU adds an update gate and reset gate but no separate cell state.

12 LSTM and GRU Implementation

The same classifier used for the RNN is reused, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

Table 4: Recurrent layer configuration – all other conditions unchanged.

13 Required Training Plots

Plot 2: Training and Validation Loss



Figure 2: Training and validation loss for RNN, LSTM and GRU.

Plot 3: Training and Validation Accuracy

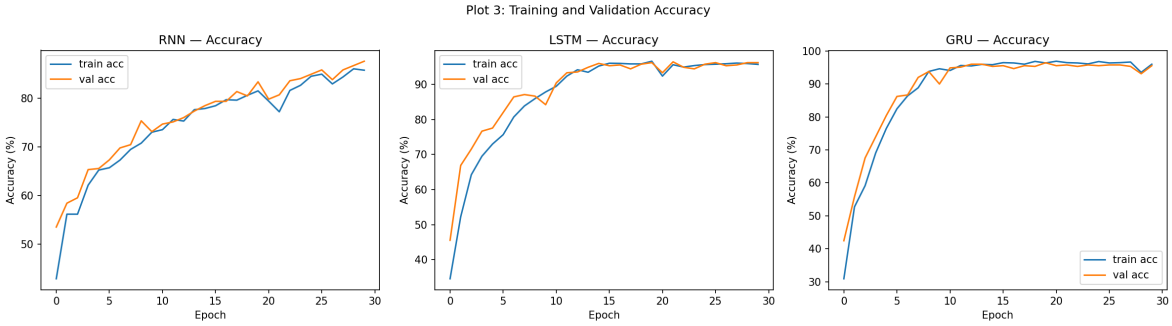


Figure 3: Training and validation accuracy for RNN, LSTM and GRU.

14 Performance Evaluation

Mandatory metrics: Accuracy, Precision, Recall, F1-score (macro-averaged), confusion matrix, number of trainable parameters, training time.

Metric	RNN	LSTM	GRU
Accuracy (%)	87.333333	96.000000	95.555556
Macro Precision (%)	87.484850	96.162436	95.603232
Macro Recall (%)	87.333333	96.000000	95.555556
Macro F1 (%)	87.330661	95.991290	95.552630
Parameters	1974.0	6006.0	4758.0
Training Time (s)	52.833251	88.548178	124.850086

Table 5: Test-set performance comparison.

15 Confusion Matrix Analysis

Plot 4: Confusion Matrix

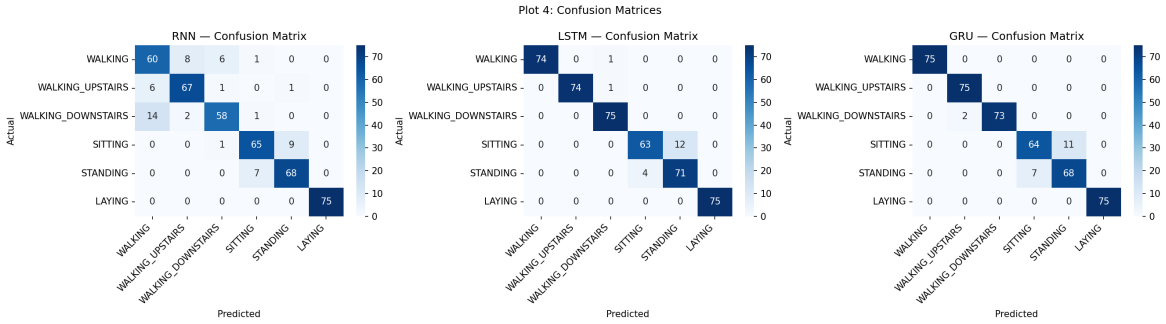


Figure 4: Confusion matrices for RNN, LSTM and GRU.

16 RNN vs. LSTM vs. GRU Comparison

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1974	6006	4758
Training time	52.8	88.5	124.9
Test F1-score	87.33	95.99	95.55

Table 6: Architectural and empirical comparison of RNN, LSTM and GRU.

Plot 5: Model Performance Comparison

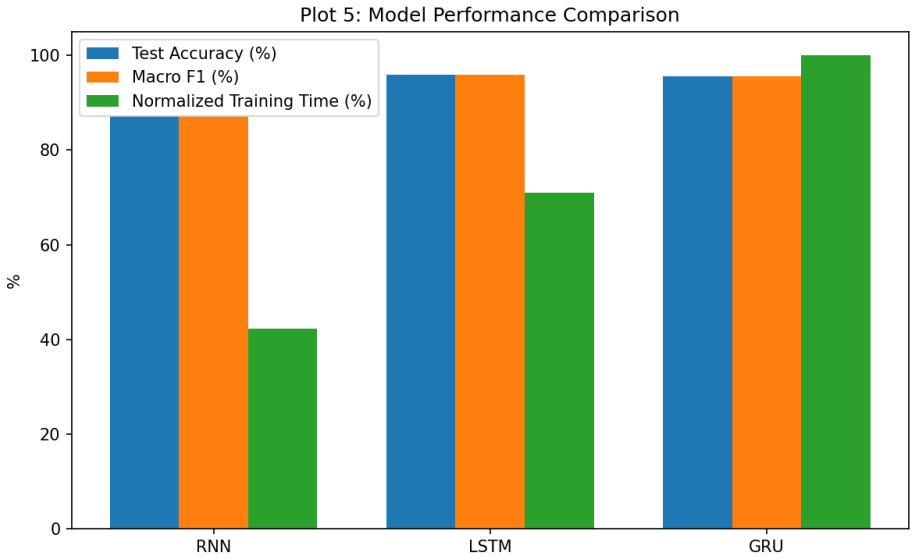


Figure 5: Bar plot comparing test accuracy, macro F1-score and normalized training time.

17 Effect of Sequence Length

Sequence lengths $T \in \{32, 64, 128\}$ are compared, truncating/constructing the input consistently.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	86.38	95.97	94.22
64	78.16	94.78	95.31
128	81.32	95.550849	95.553313

Table 7: Effect of sequence length on macro F1-score.

Plot 6: Sequence Length vs. Test F1-score

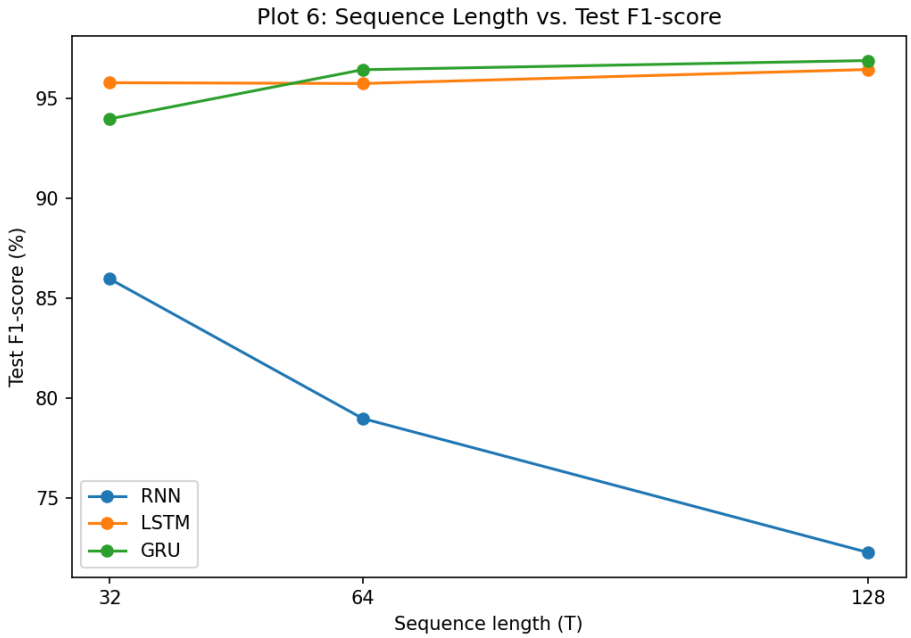
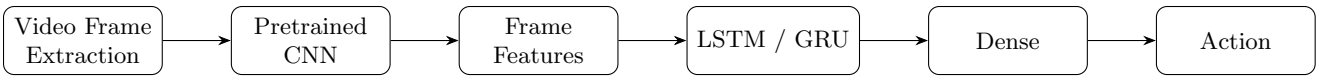


Figure 6: Test F1-score as a function of sequence length.

18 Video Understanding Using CNN and RNN

A video is a sequence of image frames. CNNs extract spatial features from individual frames, while an RNN/LSTM/GRU models the temporal relationship among those features.



Recommended dataset: a small subset of UCF101 (`https://www.crcv.ucf.edu/data/UCF101.php`), 3–5 classes (e.g. Basketball, Biking, Walking, Running, TennisSwing), a small number of videos per class.

19 Expected Video Input and Output

10 frames are sampled uniformly per video, each resized to $224 \times 224 \times 3$. A pretrained CNN (MobileNetV2) produces a feature vector of dimension D; the recurrent network receives $X \in \mathbb{R}^{10\times D}$, or $X \in \mathbb{R}^{B\times 10\times D}$ for a batch of B videos. The output is a probability vector over the selected action classes.

```
Predicted class : Basketball
Actual class    : Basketball
Confidence      : 0.xx
```

20 CNN Feature Extraction

A pretrained CNN (MobileNetV2) with the classification head removed, global average pooling applied, is used only as a feature extractor. The CNN is frozen – it is *not* trained from scratch.

21 CNN–LSTM / CNN–GRU Model

Plot 7: Video Sample Frames



Figure 7: Representative sampled frames from one video.

Plot 8: Video Training and Validation Curves

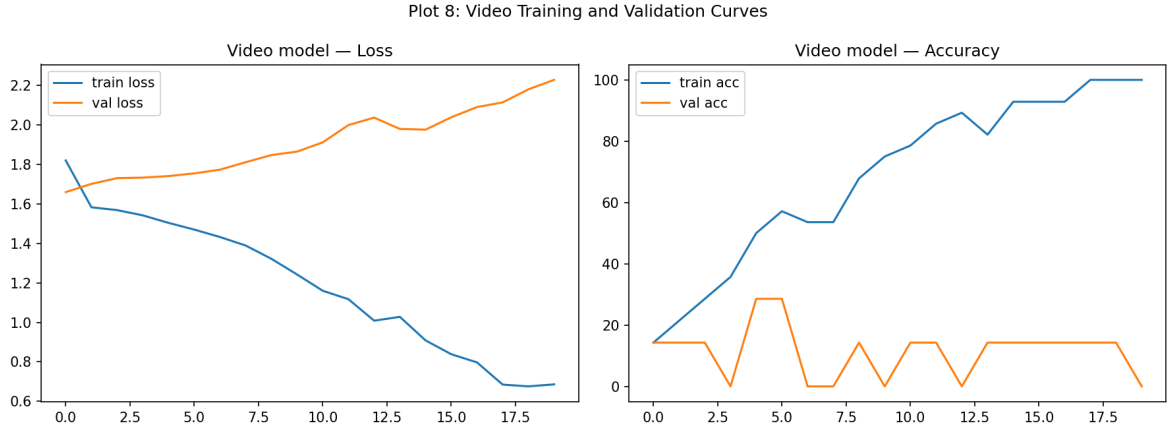


Figure 8: Training/validation accuracy and loss for the CNN-LSTM/GRU model.

Plot 9: Video Confusion Matrix

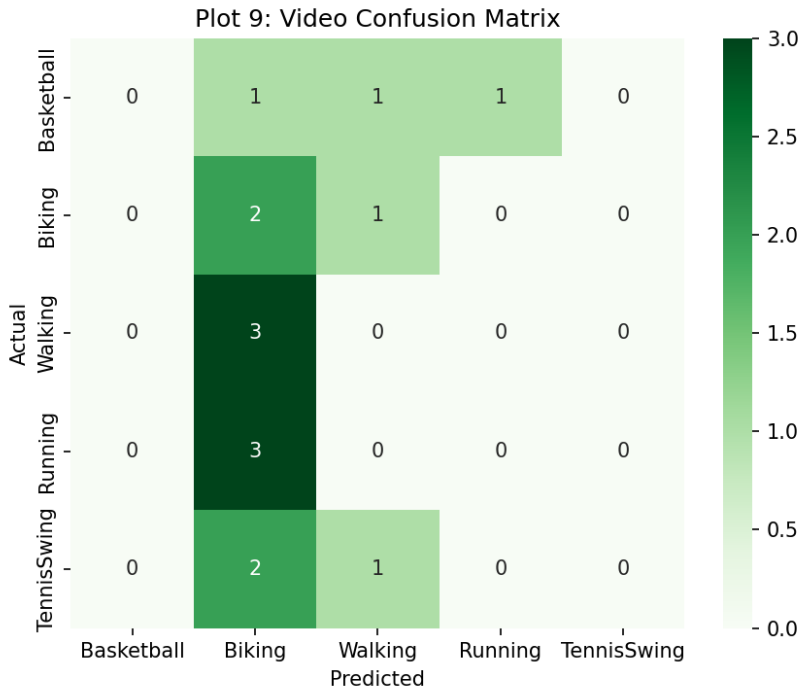


Figure 9: Confusion matrix for the video action-recognition model.

22 Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another. A synthetic reversal task is used, e.g. $[1, 4, 7, 2] \rightarrow [2, 7, 4, 1]$, generated from integers in a fixed range.



The encoder converts the input sequence into a learned representation; the decoder generates the output sequence one element at a time.

23 Expected Sequence-to-Sequence Input and Output

Example: input $[3, 8, 1, 5] \rightarrow$ expected output $[5, 1, 8, 3]$.

Sample	Input Sequence	Predicted Output
1	[np.int64(4), np.int64(2), np.int64(2), np.int64(7)]	[np.int64(7), np.int64(2), np.int64(2), np.int64(4)]
2	[np.int64(7), np.int64(8), np.int64(9), np.int64(0)]	[np.int64(0), np.int64(9), np.int64(8), np.int64(7)]
3	[np.int64(0), np.int64(9), np.int64(5), np.int64(2)]	[np.int64(2), np.int64(5), np.int64(9), np.int64(0)]
4	[np.int64(1), np.int64(5), np.int64(8), np.int64(5)]	[np.int64(5), np.int64(8), np.int64(5), np.int64(1)]
5	[np.int64(5), np.int64(8), np.int64(1), np.int64(9)]	[np.int64(9), np.int64(1), np.int64(8), np.int64(5)]

Table 8: At least five test examples of the reversal task.

24 Sequence-to-Sequence Evaluation

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}, \qquad \text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Metric	Value
Token accuracy	1.0000
Sequence accuracy	1.0000
Training loss	0.0024
Validation loss	0.0029

Table 9: Sequence-to-sequence evaluation results.

25 Consolidated Results

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	87.333333	87.484850	87.333	87.3306	1974
LSTM	96.000	96.162436	96.00000	95.9912	6006
GRU	95.555	95.603	95.555	95.552630	4758
CNN–LSTM/GRU	NaN	NaN	NaN	NaN	NaN

Table 10: Consolidated HAR task results.

Metric	Value
Token Accuracy	1.0
Sequence Accuracy	1.0

Table 11: Sequence-to-sequence task results (separate table).

26 Expected Outcome

At the end of the experiment, the student demonstrates the complete sequence-learning pipeline:

$$\text{Sequential Data} \rightarrow \text{Preprocessing} \rightarrow \text{RNN/LSTM/GRU} \rightarrow \text{Evaluation}$$

and the video-understanding pipeline:

$$\text{Video} \rightarrow \text{Frames} \rightarrow \text{CNN Features} \rightarrow \text{LSTM/GRU} \rightarrow \text{Action Prediction}$$

as well as the encoder–decoder formulation:

$$\text{Input Sequence} \rightarrow \text{Encoder} \rightarrow \text{Context} \rightarrow \text{Decoder} \rightarrow \text{Output Sequence}$$

Numerical performance values are obtained from the student's own execution and are not assumed in advance.

27 References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997.
3. Kyunghyun Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," EMNLP, 2014.
4. Anguita et al., "A Public Domain Dataset for Human Activity Recognition Using Smartphones," ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, "UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild," 2012.
6. TensorFlow Documentation: `https://www.tensorflow.org`
7. Keras Documentation: `https://keras.io`
8. Git-Hub: `https://github.com/PranavMenon07/Deep-Learning`